

Students' Ability to Distinguish AI-Generated Code from Human-Written Code

Final Clean Benchmark Dataset & Research Instrument

RESEARCHER MASTER COPY — CONTAINS ANSWER KEY — DO NOT DISTRIBUTE TO PARTICIPANTS

Domain	Language	Questions
Python – Programming & Algorithms	Python	5 (Q1–Q5)
Java – Data Structures & OOP	Java	5 (Q6–Q10)
C++ – Programming & Algorithms	C++	5 (Q11–Q15)
JavaScript – Web/Application Logic	JavaScript	5 (Q16–Q20)
SQL – Database Programming	SQL	5 (Q21–Q25)
Total	25 questions	25

Methodological note: Human/AI labels are provisional benchmark annotations until authentic human-code provenance and AI-generation records are documented. Observable coding characteristics are not treated as proof of authorship.

Question 1 — Easy

Problem Statement: Write a function `reverse_words(sentence)` that takes a string of space-separated words and returns a string with the order of the words reversed, keeping spaces single between words.

Snippet A

```
def reverse_words(sentence):  
    words = sentence.split()  
    words.reverse()  
    return " ".join(words)
```

Snippet B

```
def reverse_words(sentence):  
    words = sentence.split()  
    words = words[::-1]  
    return " ".join(words)
```

Answer Key: Snippet A: Human | Snippet B: AI

Primary Concept: String processing

Secondary Concepts: List manipulation, split/join

Expected Output / Behavior: `reverse_words("hello world python")` returns "python world hello"

Internal Classification Rationale (Researcher Only): The snippets use the same split-and-join strategy but differ in how the word order is reversed. Snippet A mutates the list in place with `.reverse()`, while Snippet B creates a reversed copy using slicing. These are observable implementation differences, not inherent indicators of authorship.

Question 2 — Easy / Moderate

Problem Statement: Write a function `second_largest(numbers)` that takes a list of integers and returns the second largest distinct integer in the list. Return `None` if no such element exists.

Snippet A

```
def second_largest(numbers):
    if len(numbers) < 2:
        return None
    first = second = float('-inf')
    for num in numbers:
        if num > first:
            second = first
            first = num
        elif num > second and num != first:
            second = num
    return second if second != float('-inf') else
None
```

Snippet B

```
def second_largest(numbers):
    unique_nums = list(set(numbers))
    if len(unique_nums) < 2:
        return None
    unique_nums.sort(reverse=True)
    return unique_nums[1]
```

Answer Key: Snippet A: Human | Snippet B: AI

Primary Concept: Searching & Filtering

Secondary Concepts: Deduplication, sorting

Expected Output / Behavior: `second_largest([10, 20, 4, 45, 99, 99])` returns 45

Internal Classification Rationale (Researcher Only): Snippet A uses a single-pass iterative search with explicit accumulator variables. Snippet B leverages built-in collections and sorting for global deduplication and ordering.

Question 3 — Moderate

Problem Statement: Write a function `is_armstrong(n)` that checks whether an n-digit positive integer is an Armstrong number.

Snippet A

```
def is_armstrong(n):
    num_str = str(n)
    power = len(num_str)
    total = 0
    temp = n
    while temp > 0:
        digit = temp % 10
        total += digit ** power
        temp //= 10
    return total == n
```

Snippet B

```
def is_armstrong(n):
    digits = [int(d) for d in str(n)]
    power = len(digits)
    return sum(d ** power for d in digits) == n
```

Answer Key: Snippet A: Human | Snippet B: AI

Primary Concept: Number operations

Secondary Concepts: Digit processing, arithmetic

Expected Output / Behavior: `is_armstrong(153)` returns True

Internal Classification Rationale (Researcher Only): Snippet A uses modulo arithmetic and integer division (``% 10``, ``//= 10``) inside a ``while`` loop. Snippet B casts the number to string, builds a list via list comprehension, and sums powers with a generator expression.

Question 4 — Moderate

Problem Statement: Write a function `find_pairs(arr, target)` that finds all unique pairs of numbers in an array that sum up to a target value.

Snippet A

```
def find_pairs(arr, target):
    pairs = []
    for i in range(len(arr)):
        for j in range(i + 1, len(arr)):
            if arr[i] + arr[j] == target:
                pair = (min(arr[i], arr[j]),
                       max(arr[i], arr[j]))
                if pair not in pairs:
                    pairs.append(pair)
    return pairs
```

Snippet B

```
def find_pairs(arr, target):
    seen = set()
    pairs = set()
    for num in arr:
        complement = target - num
        if complement in seen:
            pairs.add((min(num, complement),
                      max(num, complement)))
        seen.add(num)
    return list(pairs)
```

Answer Key: Snippet A: Human | Snippet B: AI

Primary Concept: Lists & Hash Sets

Secondary Concepts: Nested loops, hashing

Expected Output / Behavior: Returns the unique pairs summing to the target; pair order may vary.

Internal Classification Rationale (Researcher Only): Snippet A uses nested index loops and list membership checks. Snippet B uses a hash set for complement lookups and set-based deduplication. The difference is an observable algorithmic choice; neither approach is inherently associated with Human or AI authorship.

Question 5 — Moderate

Problem Statement: Write a function `generate_primes(limit)` that returns all prime numbers less than or equal to `limit`.

Snippet A

```
def generate_primes(limit):
    primes = []
    for n in range(2, limit + 1):
        is_prime = True
        for p in primes:
            if p * p > n:
                break
            if n % p == 0:
                is_prime = False
                break
        if is_prime:
            primes.append(n)
    return primes
```

Snippet B

```
def generate_primes(limit):
    if limit < 2:
        return []
    flags = [True] * (limit + 1)
    flags[0] = flags[1] = False
    for n in range(2, int(limit ** 0.5) + 1):
        if flags[n]:
            for multiple in range(n * n, limit + 1, n):
                flags[multiple] = False
    return [i for i in range(2, limit + 1) if flags[i]]
```

Answer Key: Snippet A: AI | Snippet B: Human

Primary Concept: Number algorithms

Secondary Concepts: Prime generation, algorithmic trade-offs

Expected Output / Behavior: `generate_primes(10)` returns `[2, 3, 5, 7]`.

Internal Classification Rationale (Researcher Only): Snippet A tests each candidate using previously found primes. Snippet B marks composite numbers with a sieve. Both are valid prime-generation approaches with different algorithmic trade-offs; the implementation style itself is not treated as proof of authorship.

Question 6 — Easy

Problem Statement: Write a static method `isSorted(int[] arr)` that returns true if an array of integers is sorted. Assumption: `arr` is guaranteed to be non-null.

Snippet A

```
public class ArrayUtils {
    public static boolean isSorted(int[] arr) {
        if (arr == null || arr.length <= 1) {
            return true;
        }
        for (int i = 0; i < arr.length - 1; i++) {
            if (arr[i] > arr[i + 1]) {
                return false;
            }
        }
        return true;
    }
}
```

Snippet B

```
public class Solution {
    public static boolean isSorted(int[] arr) {
        for (int i = 0; i < arr.length - 1; i++) {
            if (arr[i] > arr[i + 1]) {
                return false;
            }
        }
        return true;
    }
}
```

Answer Key: Snippet A: Human | Snippet B: AI

Primary Concept: Arrays

Secondary Concepts: Array traversal, boundary handling

Expected Output / Behavior: `isSorted(new int[]{1, 2, 4, 7, 9})` returns true

Internal Classification Rationale (Researcher Only): Snippet A includes an explicit guard for null or very short arrays, while Snippet B relies on the loop bounds to handle short arrays. The stated assumption makes the two implementations equivalent for valid inputs. This is an observable edge-case handling difference, not an authorship rule.

Question 7 — Easy / Moderate

Problem Statement: Write a method `countCharacterFrequency(String input)` that returns a Map containing each alphabetic character in the string and its frequency. Assumption: input contains ASCII alphabetic characters (A-Z/a-z); other characters may be ignored.

Snippet A

```
import java.util.HashMap;
import java.util.Map;
public class CharFrequency {
    public static Map<Character, Integer> countCharacterFrequency(String input) {
        Map<Character, Integer> counts = new HashMap<>();
        String cleaned = input.toLowerCase();
        for (int i = 0; i < cleaned.length(); i++) {
            char c = cleaned.charAt(i);
            if (c >= 'a' && c <= 'z') {
                counts.put(c, counts.getOrDefault(c, 0) + 1);
            }
        }
        return counts;
    }
}
```

Snippet B

```
import java.util.HashMap;
import java.util.Map;
public class CharFrequency {
    public static Map<Character, Integer> countCharacterFrequency(String input) {
        Map<Character, Integer> map = new HashMap<>();
        for (char ch : input.toLowerCase().toCharArray()) {
            if (Character.isLetter(ch)) {
                map.merge(ch, 1, Integer::sum);
            }
        }
        return map;
    }
}
```

Answer Key: Snippet A: Human | Snippet B: AI

Primary Concept: HashMap & Collections

Secondary Concepts: HashMap, character processing

Expected Output / Behavior: Returns map with lowercase letter frequencies.

Internal Classification Rationale (Researcher Only): Snippet A uses index traversal with `charAt(i)`, an ASCII range check, and `getOrDefault`. Snippet B uses `toCharArray()`, `Character.isLetter()`, and `map.merge`. These are observable implementation differences, not authorship indicators.

Question 8 — Moderate

Problem Statement: Write a method `areParenthesesBalanced(String expr)` using a Stack to check if a string has balanced brackets.

Snippet A

```
import java.util.ArrayDeque;
import java.util.Deque;
public class BalanceChecker {
    public static boolean areParenthesesBalanced(String expr) {
        Deque<Character> stack = new ArrayDeque<>();
        for (char c : expr.toCharArray()) {
            if (c == '(' || c == '{' || c == '[') {
                stack.push(c);
            } else if (c == ')' || c == '}' || c == ']') {
                if (stack.isEmpty()) return false;
                char top = stack.pop();
                if ((c == ')' && top != '(') ||
                    (c == '}' && top != '{') ||
                    (c == ']' && top != '[')) return false;
            }
        }
        return stack.isEmpty();
    }
}
```

Snippet B

```
import java.util.Stack;
public class BalanceChecker {
    public static boolean areParenthesesBalanced(String expr) {
        Stack<Character> st = new Stack<>();
        for (int i = 0; i < expr.length(); i++) {
            char ch = expr.charAt(i);
            if (ch == '(' || ch == '{' || ch == '[') {
                st.push(ch);
            } else if (ch == ')' || ch == '}' || ch == ']') {
                if (st.isEmpty()) return false;
                char open = st.pop();
                if (ch == ')' && open != '(') return false;
                if (ch == '}' && open != '{') return false;
                if (ch == ']' && open != '[') return false;
            }
        }
        return st.isEmpty();
    }
}
```

Answer Key: Snippet A: AI | Snippet B: Human

Primary Concept: Stack Data Structure

Secondary Concepts: Stack/Deque, delimiter matching

Expected Output / Behavior: `areParenthesesBalanced("{{()}}")` returns true

Internal Classification Rationale (Researcher Only): Snippet A uses `Deque` with `ArrayDeque`. Snippet B uses legacy `java.util.Stack` and explicit `if/else` return logic.

Question 9 — Moderate

Problem Statement: Implement a simple BankAccount class with deposit, withdraw, and interest logic.

Snippet A

```
public class BankAccount {
    private String accountNumber;
    private double balance;
    public BankAccount(String accountNumber,
double initialBalance) {
        this.accountNumber = accountNumber;
        this.balance = initialBalance;
    }
    public void deposit(double amount) {
        if (amount > 0) balance += amount;
    }
    public boolean withdraw(double amount) {
        if (amount > 0 && balance >= amount) {
            balance -= amount;
            return true;
        }
        return false;
    }
    public void applyInterest(double
ratePercentage) {
        if (ratePercentage > 0) {
            balance += balance * (ratePercentage /
100.0);
        }
    }
}
```

Snippet B

```
public class BankAccount {
    String accNum;
    double balance;
    public BankAccount(String accNum, double
balance) {
        this.accNum = accNum;
        this.balance = balance;
    }
    public void deposit(double amt) {
        if (amt > 0) balance += amt;
    }
    public boolean withdraw(double amt) {
        if (amt > 0 && amt <= balance) {
            balance -= amt;
            return true;
        }
        return false;
    }
    public void applyInterest(double rate) {
        if (rate > 0) {
            double interest = balance * (rate /
100.0);
            balance += interest;
        }
    }
}
```

Answer Key: Snippet A: AI | Snippet B: Human

Primary Concept: Object-Oriented Programming

Secondary Concepts: Encapsulation, state management

Expected Output / Behavior: Class encapsulates balance state and applies calculations correctly.

Internal Classification Rationale (Researcher Only): Snippet A uses private fields and explicit validation checks. Snippet B uses package-visible fields and different identifier names while preserving the same deposit, withdrawal, and interest state behavior.

Question 10 — Moderate / Challenging

Problem Statement: Write a method reverseList(ListNode head) that reverses a singly linked list iteratively.

Snippet A

```
public class Solution {
    public ListNode reverseList(ListNode head) {
        ListNode prev = null;
        ListNode curr = head;
        while (curr != null) {
            ListNode nextTemp = curr.next;
            curr.next = prev;
            prev = curr;
            curr = nextTemp;
        }
        return prev;
    }
}
```

Snippet B

```
public class LinkedListOps {
    public static ListNode reverseList(ListNode
head) {
    if (head == null || head.next == null)
return head;
    ListNode p1 = null;
    ListNode p2 = head;
    ListNode p3 = null;
    while (p2 != null) {
        p3 = p2.next;
        p2.next = p1;
        p1 = p2;
        p2 = p3;
    }
    return p1;
}
}
```

Answer Key: Snippet A: AI | Snippet B: Human

Primary Concept: Linked List

Secondary Concepts: Pointers, linked-list traversal

Expected Output / Behavior: Input 1 -> 2 -> 3 -> null returns 3 -> 2 -> 1 -> null.

Internal Classification Rationale (Researcher Only): Snippet A uses semantic pointer identifiers (`prev`, `curr`, `nextTemp`). Snippet B uses generic indexed pointer identifiers (`p1`, `p2`, `p3`) and explicit short-circuit checks.

Question 11 — Easy

Problem Statement: Write a C++ function findMaxAndIndex that returns the maximum value and index as an std::pair
Assumption: the input vector is non-empty.

Snippet A

```
#include <vector>
#include <utility>
std::pair<int, std::size_t> findMaxAndIndex(const std::vector<int>& arr) {
    int maxVal = arr[0];
    std::size_t maxIdx = 0;
    for (std::size_t i = 1; i < arr.size(); ++i) {
        if (arr[i] > maxVal) {
            maxVal = arr[i];
            maxIdx = i;
        }
    }
    return {maxVal, maxIdx};
}
```

Snippet B

```
#include <vector>
#include <algorithm>
#include <utility>
std::pair<int, std::size_t> findMaxAndIndex(const std::vector<int>& arr) {
    auto it = std::max_element(arr.begin(), arr.end());
    std::size_t index = std::distance(arr.begin(), it);
    return std::make_pair(*it, index);
}
```

Answer Key: Snippet A: Human | Snippet B: AI

Primary Concept: Arrays & Vectors

Secondary Concepts: Vectors, max_element, indexing

Expected Output / Behavior: Input {3, 9, 2, 5} returns std::pair(9, 1).

Internal Classification Rationale (Researcher Only): Snippet A uses explicit loop tracking. Snippet B uses STL algorithms (`std::max\_element` and `std::distance`).

Question 12 — Easy / Moderate

Problem Statement: Write a C++ function `binarySearch` that performs binary search on a sorted vector. Assumption: the input vector is non-empty and sorted in ascending order.

Snippet A

```
#include <vector>
int binarySearch(const std::vector<int>& nums, int
target) {
    int low = 0;
    int high = nums.size() - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

Snippet B

```
#include <vector>
int binarySearch(const std::vector<int>& arr, int
target) {
    int l = 0;
    int r = arr.size() - 1;
    while (l <= r) {
        int mid = (l + r) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) {
            l = mid + 1;
        } else {
            r = mid - 1;
        }
    }
    return -1;
}
```

Answer Key: Snippet A: AI | Snippet B: Human

Primary Concept: Searching Algorithms

Secondary Concepts: Loop invariants, midpoint calculation

Expected Output / Behavior: Input `{1, 3, 5, 7, 9}`, 7 returns 3.

Internal Classification Rationale (Researcher Only): Snippet A calculates midpoint via `low + (high - low) / 2`. Snippet B uses `(l + r) / 2` and abbreviated index names.

Question 13 — Moderate

Problem Statement: Write a C++ function `bubbleSort(std::vector<int>& arr)` that sorts an integer vector in ascending order. Assumption: the input vector is non-empty.

Snippet A

```
#include <vector>
void bubbleSort(std::vector<int>& arr) {
    int n = static_cast<int>(arr.size());
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

Snippet B

```
#include <vector>
#include <utility>
void bubbleSort(std::vector<int>& arr) {
    std::size_t n = arr.size();
    bool swapped;
    for (std::size_t i = 0; i + 1 < n; ++i) {
        swapped = false;
        for (std::size_t j = 0; j + 1 < n - i; ++j) {
            if (arr[j] > arr[j + 1]) {
                std::swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }
        if (!swapped) break;
    }
}
```

Answer Key: Snippet A: Human | Snippet B: AI

Primary Concept: Sorting Algorithms

Secondary Concepts: Nested loops, swapping

Expected Output / Behavior: Sorts vector in place into ascending order.

Internal Classification Rationale (Researcher Only): Snippet A uses manual temporary swapping variables. Snippet B uses `std::swap`, `size_t` typing, and pass optimization flags.

Question 14 — Moderate

Problem Statement: Write a C++ function `transposeMatrix` that returns the $N \times M$ transpose of an $M \times N$ matrix.

Assumption: the input matrix is non-empty.

Snippet A

```
#include <vector>
std::vector<std::vector<int>> transposeMatrix(
    const std::vector<std::vector<int>>& matrix) {
    if (matrix.empty()) return {};
    std::size_t rows = matrix.size();
    std::size_t cols = matrix[0].size();
    std::vector<std::vector<int>> result(cols,
    std::vector<int>(rows));
    for (std::size_t i = 0; i < rows; ++i) {
        for (std::size_t j = 0; j < cols; ++j) {
            result[j][i] = matrix[i][j];
        }
    }
    return result;
}
```

Snippet B

```
#include <vector>
std::vector<std::vector<int>> transposeMatrix(
    const std::vector<std::vector<int>>& matrix) {
    int r = matrix.size();
    int c = matrix[0].size();
    std::vector<std::vector<int>> trans;
    for (int i = 0; i < c; i++) {
        std::vector<int> row;
        for (int j = 0; j < r; j++) {
            row.push_back(matrix[j][i]);
        }
        trans.push_back(row);
    }
    return trans;
}
```

Answer Key: Snippet A: AI | Snippet B: Human

Primary Concept: Matrix Operations

Secondary Concepts: Vectors, dimensions

Expected Output / Behavior: Input `[[1,2],[3,4],[5,6]]` returns `[[1,3,5],[2,4,6]]`.

Internal Classification Rationale (Researcher Only): Snippet A pre-allocates vector dimensions upfront. Snippet B constructs dynamic inner vectors using `push_back`.

Question 15 —Moderate / Challenging

Problem Statement: Write a C++ function climbStairs(int n) that returns distinct ways to climb n steps taking 1 or 2 steps.
Assumption: n is a positive integer (n >= 1).

Snippet A

```
#include <vector>
int climbStairs(int n) {
    if (n <= 2) return n;
    std::vector<int> dp(n + 1);
    dp[1] = 1;
    dp[2] = 2;
    for (int i = 3; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }
    return dp[n];
}
```

Snippet B

```
int climbStairs(int n) {
    if (n <= 2) return n;
    int first = 1;
    int second = 2;
    int current = 0;
    for (int i = 3; i <= n; ++i) {
        current = first + second;
        first = second;
        second = current;
    }
    return current;
}
```

Answer Key: Snippet A: Human | Snippet B: AI

Primary Concept: Dynamic Programming

Secondary Concepts: Dynamic programming, space optimization

Expected Output / Behavior: climbStairs(5) returns 8.

Internal Classification Rationale (Researcher Only): Snippet A constructs a dynamic programming vector table (O(N) space). Snippet B optimizes auxiliary memory using scalar state variables (O(1) space).

Question 16 — Easy

Problem Statement: Write a JavaScript function `removeDuplicates(arr)` that returns unique elements preserving order.

Snippet A

```
function removeDuplicates(arr) {  
  return Array.from(new Set(arr));  
}
```

Snippet B

```
function removeDuplicates(arr) {  
  let result = [];  
  for (let i = 0; i < arr.length; i++) {  
    if (!result.includes(arr[i])) {  
      result.push(arr[i]);  
    }  
  }  
  return result;  
}
```

Answer Key: Snippet A: Human | Snippet B: AI

Primary Concept: Arrays & Deduplication

Secondary Concepts: Set, order preservation

Expected Output / Behavior: `removeDuplicates([1, 2, 2, 3, 4, 4, 1])` returns `[1, 2, 3, 4]`.

Internal Classification Rationale (Researcher Only): Snippet A uses Set-based deduplication with `Array.from`. Snippet B builds the result with an explicit loop and includes-based membership check. These are observable implementation differences, not inherent authorship indicators.

Question 17 — Moderate

Problem Statement: Write a JavaScript function `arrayToMap(users)` that converts an array of user objects into an object keyed by user id, with each value set to the user's name.

Snippet A

```
function arrayToMap(users) {  
  let map = {};  
  users.forEach(user => {  
    map[user.id] = user.name;  
  });  
  return map;  
}
```

Snippet B

```
function arrayToMap(users) {  
  return users.reduce((acc, user) => {  
    acc[user.id] = user.name;  
    return acc;  
  }, {});  
}
```

Answer Key: Snippet A: AI | Snippet B: Human

Primary Concept: Objects & Data Manipulation

Secondary Concepts: Objects, iteration

Expected Output / Behavior: Input `[[{id: 1, name: 'Alice'}]]` returns `{1: 'Alice'}`.

Internal Classification Rationale (Researcher Only): Snippet A mutates a local object via `.forEach()`. Snippet B uses `.reduce()` functional reduction.

Question 18 — Moderate

Problem Statement: Write an async function `retryFetch(fetchFn, maxRetries)` that retries execution up to `maxRetries` times on error. Assumption: `maxRetries` is a positive integer (`maxRetries >= 1`).

Snippet A

```
async function retryFetch(fetchFn, maxRetries) {
  for (let attempt = 1; attempt <= maxRetries; attempt++) {
    try {
      return await fetchFn();
    } catch (error) {
      if (attempt === maxRetries) {
        throw error;
      }
    }
  }
}
```

Snippet B

```
async function retryFetch(fetchFn, maxRetries) {
  let count = 0;
  while (count < maxRetries) {
    try {
      let res = await fetchFn();
      return res;
    } catch (err) {
      count++;
      if (count >= maxRetries) {
        throw err;
      }
    }
  }
}
```

Answer Key: Snippet A: AI | Snippet B: Human

Primary Concept: Async Control Flow

Secondary Concepts: Promises, error handling

Expected Output / Behavior: Resolves on success; throws final error after maximum retries.

Internal Classification Rationale (Researcher Only): Snippet A tracks loop counter state in `for` loop declarations. Snippet B uses a manual counter inside a `while` loop updated within `catch`.

Question 19 — Moderate

Problem Statement: Write a function `calculateCartTotal(cart, discountCode)` processing item subtotals and applying discounts.

Snippet A

```
function calculateCartTotal(cart, discountCode) {
  let total = 0;
  for (let item of cart) {
    total += item.price * item.quantity;
  }
  if (discountCode === "TEN") {
    total = total * 0.9;
  } else if (discountCode === "TWENTY") {
    total = total * 0.8;
  }
  return Math.round(total * 100) / 100;
}
```

Snippet B

```
function calculateCartTotal(cart, discountCode) {
  const discounts = { TEN: 0.10, TWENTY: 0.20 };
  const subtotal = cart.reduce((sum, item) =>
    sum + item.price * item.quantity, 0);
  const discountRate = discounts[discountCode] ||
0;
  const finalTotal = subtotal * (1 -
discountRate);
  return Math.round(finalTotal * 100) / 100;
}
```

Answer Key: Snippet A: Human | Snippet B: AI

Primary Concept: Application Logic

Secondary Concepts: Discount lookup, rounding

Expected Output / Behavior: Sums items, applies discount, and returns float rounded to 2 decimal places.

Internal Classification Rationale (Researcher Only): Snippet A uses `for...of` iteration, explicit conditional discount logic, and `Math.round`. Snippet B uses `reduce()`, an object-based discount map, and the same `Math.round` rounding expression.

Question 20 — Moderate / Challenging

Problem Statement: Write a JavaScript function `flattenArray(arr)` that flattens arbitrarily nested sub-arrays.

Snippet A

```
function flattenArray(arr) {
  return arr.reduce((acc, val) =>
    Array.isArray(val) ?
    acc.concat(flattenArray(val)) : acc.concat(val),
    []);
}
```

Snippet B

```
function flattenArray(arr) {
  let flat = [];
  function doFlatten(list) {
    for (let i = 0; i < list.length; i++) {
      if (Array.isArray(list[i])) {
        doFlatten(list[i]);
      } else {
        flat.push(list[i]);
      }
    }
  }
  doFlatten(arr);
  return flat;
}
```

Answer Key: Snippet A: AI | Snippet B: Human

Primary Concept: Array Flattening & Recursion

Secondary Concepts: Recursion, nested arrays

Expected Output / Behavior: `flattenArray([1, [2, [3, 4], 5]])` returns `[1, 2, 3, 4, 5]`.

Internal Classification Rationale (Researcher Only): Snippet A uses functional expression recursion inside ``reduce``. Snippet B uses internal procedural helper recursion mutating a closure array.

Question 21 — Easy

Problem Statement: Find all employees working in 'Sales' whose salary is higher than at least one employee in 'Marketing'.

Snippet A

```
SELECT employee_id, first_name, last_name
FROM employees
WHERE department = 'Sales'
  AND salary > ANY (
    SELECT salary
    FROM employees
    WHERE department = 'Marketing'
  );
```

Snippet B

```
SELECT employee_id, first_name, last_name
FROM employees
WHERE department = 'Sales'
  AND salary > (
    SELECT MIN(salary)
    FROM employees
    WHERE department = 'Marketing'
  );
```

Answer Key: Snippet A: Human | Snippet B: AI

Primary Concept: SELECT + WHERE

Secondary Concepts: ANY, MIN, subqueries

Expected Output / Behavior: Identical rows matching Sales employees earning above the minimum Marketing salary.

Internal Classification Rationale (Researcher Only): Snippet A uses the SQL quantifier keyword `> ANY` over a subquery result. Snippet B replaces quantifiers with explicit subquery scalar aggregation (`> MIN()`).

Question 22 — Easy / Moderate

Problem Statement: Retrieve the top 5 most expensive products in category_id = 3. Break ties deterministically using product_id in ascending order.

Snippet A

```
SELECT product_name, price
FROM products
WHERE category_id = 3
ORDER BY price DESC, product_id ASC
LIMIT 5;
```

Snippet B

```
SELECT product_name, price
FROM products p1
WHERE category_id = 3
  AND 5 > (
    SELECT COUNT(*)
    FROM products p2
    WHERE p2.category_id = 3
      AND (p2.price > p1.price
        OR (p2.price = p1.price AND
          p2.product_id < p1.product_id))
  )
ORDER BY price DESC, product_id ASC;
```

Answer Key: Snippet A: AI | Snippet B: Human

Primary Concept: ORDER BY / Filtering

Secondary Concepts: LIMIT, correlated subquery, tie handling

Expected Output / Behavior: Guaranteed identical top 5 product rows in category 3, even under price ties, due to strict total ordering on product_id.

Internal Classification Rationale (Researcher Only): Snippet A uses standard relational sorting and row restriction (`ORDER BY ... LIMIT`). Snippet B uses relational rank filtering via a correlated subquery counting preceding items under a strict total order `(price DESC, product\_id ASC)`.

Question 23 — Moderate

Problem Statement: Calculate total budget and average salary for departments with more than 5 employees.

Snippet A

```
SELECT department_id,  
       SUM(salary) AS total_budget,  
       AVG(salary) AS avg_salary  
FROM employees  
GROUP BY department_id  
HAVING COUNT(*) > 5;
```

Snippet B

```
SELECT department_id,  
       SUM(salary) AS total_budget,  
       AVG(salary) AS avg_salary  
FROM employees  
WHERE department_id IN (  
  SELECT department_id  
  FROM employees  
  GROUP BY department_id  
  HAVING COUNT(*) > 5  
)  
GROUP BY department_id;
```

Answer Key: Snippet A: AI | Snippet B: Human

Primary Concept: Aggregations & GROUP BY

Secondary Concepts: SUM, AVG, HAVING, subquery

Expected Output / Behavior: Grouped metrics strictly for departments containing over 5 records.

Internal Classification Rationale (Researcher Only): Snippet A applies group filtering directly in the main query via `HAVING`. Snippet B pre-filters qualifying departments in a subquery `WHERE department\_id IN (...)` before applying main grouping.

Question 24 — Moderate

Problem Statement: List all customers and order dates for 2024 orders, including customers with no orders.

Snippet A

```
SELECT c.customer_name, o.order_date
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
    AND o.order_date >= '2024-01-01'
    AND o.order_date < '2025-01-01';
```

Snippet B

```
SELECT c.customer_name, o.order_date
FROM customers c
LEFT JOIN (
    SELECT order_id, customer_id, order_date
    FROM orders
    WHERE order_date >= '2024-01-01'
    AND order_date < '2025-01-01'
) o ON c.customer_id = o.customer_id;
```

Answer Key: Snippet A: Human | Snippet B: AI

Primary Concept: Table JOINS

Secondary Concepts: LEFT JOIN, date filtering

Expected Output / Behavior: All customer names matched with 2024 order dates (or NULL).

Internal Classification Rationale (Researcher Only): Snippet A includes date range predicates directly inside the `LEFT JOIN ... ON` clause. Snippet B joins customers against a pre-filtered derived table subquery containing date conditions.

Question 25 — Moderate / Challenging

Problem Statement: Find all employees whose salary is higher than the average salary of their respective department.

Snippet A

```
SELECT e.employee_id, e.department_id, e.salary
FROM employees e
WHERE e.salary > (
    SELECT AVG(emp.salary)
    FROM employees emp
    WHERE emp.department_id = e.department_id
);
```

Snippet B

```
SELECT e.employee_id, e.department_id, e.salary
FROM employees e
JOIN (
    SELECT department_id, AVG(salary) AS avg_sal
    FROM employees
    GROUP BY department_id
) d ON e.department_id = d.department_id
WHERE e.salary > d.avg_sal;
```

Answer Key: Snippet A: AI | Snippet B: Human

Primary Concept: Subqueries

Secondary Concepts: Correlated subquery, derived table JOIN

Expected Output / Behavior: Employees earning strictly above their department's calculated average.

Internal Classification Rationale (Researcher Only): Snippet A evaluates a correlated subquery for each outer row. Snippet B pre-aggregates department averages into a derived view and performs an explicit relational `JOIN` on `department\_id`.

Research Metadata & Provenance

Code Provenance

Q.No	Human Code Source	AI Code Source	AI Tool/Model	Generation Date	Status
Q1–Q25	Pending — authentic human-written code to be collected	Pending	Pending	Pending	Pending

Expert Validation

Q.No	Functional Equivalence	Correctness	Complexity Comparable	Expert Review	Status
Q1–Q25	Pending	Pending	Pending	Pending	Pending

Quality Audit & Revision Log

Check Item	Result	Detail
25 questions	PASS	Exactly 25 benchmark items present.
5 questions per domain	PASS	Python (5), Java (5), C++ (5), JavaScript (5), SQL (5).
All snippets syntactically valid	PASS*	Source rebuilt cleanly; representative syntax checked.
Functional equivalence internally checked	PASS*	Checked against stated assumptions and representative inputs.
No obvious Human/AI stereotype	PASS	Rationales describe observable implementation differences rather than authorship stereotypes.
SQL Q21–Q25 verified	PASS	Equivalent structural query strategies with deterministic tie/date handling.
Q25 corrected	PASS	Both snippets use department-level averages.
Human/AI positions balanced	PASS	Snippet A = Human (13); Snippet A = AI (12).
Provenance documented/pending	PASS	Authentic human provenance and AI generation records remain pending.
Expert validation documented/pending	PASS	Expert review remains pending.

*Internal verification only; external expert validation has not yet been completed.

Revision Log

Item	Change
Q1	Difficulty reset to Easy; snippets kept visually comparable.
Q4	Expected behavior explicitly notes that pair order may vary.
Q5	Replaced the redundant anagram item with prime-number generation.
Q6	Null-input assumption retained; snippets aligned for valid inputs.
Q7	ASCII-input assumption retained; researcher rationale restored.
Q9	Return type and validation behavior aligned between snippets.
Q16	Set-based implementation is Human to break the idiomatic-style pattern.
Q17	reduce()-based implementation is Human to break the idiomatic-style pattern.
Q19	Rounding standardized to Math.round in both snippets.
Q22	Deterministic product_id tie-breaker retained.
Q23–Q25	SQL equivalence/formatting corrections retained.